

write-review-and-repair

A User's Manual

The examine-then-repair pipeline, end to end.

Examine it, then repair it.

Last revised: September 11, 2026

torsor lab

torsor.org

Contents

Preface: Fixing Is the Other Half	iii
1 Two Stages, One Handoff	1
The shape	1
The handoff is on disk	2
Written is not accepted	3
Reuse, don't rebuild	3
2 Running It	4
Stage one, in brief	4
What stage two checks before starting	5
Triage, before any mathematics	5
Working out the repairs	6
The audit	7
Apply, propose, or escalate	7
Two builds	9
Corrections flow back	10
3 Reading What Comes Back	11
What is on disk	12
The three states	13
Answering by tag	13

What the markup cannot do	14
What cannot be repaired at all	15
If a stage stops short	15
A Quick Reference	16
Invocation	16
The pipeline	17
Stage two’s preconditions	17
Dispositions	17
Markup states	18
Markup commands	18
Markup rules that came out of real use	19
Builds	20
Outputs	20
Adopting a proposal later	21
Known limits	21

Preface: Fixing Is the Other Half

Let's be honest with each other. Finding what's wrong with a paper is the part everybody talks about. Working out what would fix it is the part that takes the afternoon.

You've found the gap in Section 4. Now: is it fillable? With what hypothesis? Does adding that hypothesis break Example 3.7? Does it change what Theorem 1.1 is claiming? And once you've decided all that, where exactly in fifty pages of someone else's LaTeX do the sentences go?

`write-review-and-repair` runs both halves.

First, `write-critical-guide` examines the paper and produces a graded, located ledger of issues. Then `write-paper-repairs` takes that ledger, works out the mathematics that closes each gap, audits every repair adversarially, and writes the ones the paper itself forces into a copy of the authors' own source as tracked changes.

What comes back is a corrected source that compiles two ways — changes marked up for review, or silently in place — plus the repairs that were worked out but not adopted, shown in position so you can browse them, plus a report of what no repair reaches.

Nothing is applied silently, and nothing is decided for you. A repair goes in only when it is *forced*: when there is exactly one way to make

the text consistent with what the paper already demonstrably contains. Everything else is proposed, tagged, and left for you to accept or reject one at a time.

The pipeline itself is thin. It sequences two skills that already run unattended and already share a handoff on disk, and it carries no examination or repair judgment of its own. That's deliberate — the interesting parts belong to the stages, and the orchestrator's whole job is to run them in order and check that the handoff between them is real.

Three chapters. Chapter 1 is the two-stage shape and why the handoff lives on disk. Chapter 2 is running it, with the repair stage covered at working depth — how it triages, what it will and won't write into your paper, and why it applies so few. Chapter 3 is how to read what comes back and how to adopt more of it. The appendix has the markup commands and the build lines.

The examination stage has its own manual, and this one doesn't repeat it. If you want the sweeps, the refutation pass, and the ledger schema, read *write-critical-guide: A User's Manual* alongside this.

Two Stages, One Handoff

Examine, then repair. Two skills, run in order, talking to each other through a file.

The shape

Stage	What it does
write-critical-guide	reads the paper and produces a critical-guide/ package in the paper's folder: a graded, located issue ledger, annotated sources, and a typeset guide. It surfaces and grades issues; it reaches no verdict.
write-paper-repairs	consumes that package, works out the mathematics, and writes the forced repairs into a copy of the paper's own source as tracked changes. It shows the repairs it did not adopt in position, and reports what cannot be repaired. It repairs; it does not decide what the paper should claim.

Each stage runs unattended and each is useful on its own. The pipeline exists because running them back to back is what you usually want, and because the handoff between them has a failure mode worth checking every time.

The handoff is on disk

Stage one writes `critical-guide/issues.yaml`. Stage two reads it. That's the entire interface.

Which means the orchestrator holds almost no state. Once stage one returns, the only thing carried into stage two is *where the package landed*. Everything else can be discarded, and that is what keeps the pipeline light enough to run over a long paper without the context of the examination crowding out the repairs.

It also means the stages compose in other ways. You can examine a paper today and repair it next month. You can examine it, read the guide, decide the findings are wrong, and never repair it at all. You can point stage two at a package that stage one produced in a different session on a different machine.

Because the handoff is a file, it can be stale. If the authors post a new version, or the source is edited between the two stages, the ledger's anchors stop matching and the repairs get aimed at passages that no longer exist. Stage two checks this before doing anything and stops if it fails — but the check belongs to stage two, and running the pipeline end to end is what makes the ledger as fresh as it will ever be.

Between the stages, the pipeline confirms two things: that stage one's package directory is there, and that it holds `issues.yaml`. If either is missing or partial it stops and reports rather than running repairs against a half-written ledger. Repairs aimed at issues that were never recorded are worse than no repairs.

Written is not accepted

The one framing that matters, and the one most likely to be lost when the pipeline is run unattended and you read the result three days later.

Stage two's output is a *marked-up* build. Proposed changes are shown in position. Nothing has been applied to the paper. Running both stages back to back does not adopt anything — the person reading the result decides, tag by tag.

Even the repairs that *are* applied to the working copy are applied to a copy. The original source is never modified.

Reuse, don't rebuild

If a `critical-guide/` package already exists for the paper, the pipeline uses it and goes straight to repairs. Examination is the expensive half and there is no reason to pay for it twice.

Say explicitly that you want it regenerated if the paper has changed since the package was made — otherwise the old ledger is what gets repaired against, and its anchors are exactly what will fail the staleness check.

Running It

```
/write-review-and-repair ~/refereeing/  
descent.pdf  
/write-review-and-repair arXiv:0000.00000  
/write-review-and-repair ~/refereeing/  
descent/
```

It runs unattended, end to end. Both stages are invoked with their questions pre-answered, so neither stops to confirm the reader, the location, the outline, or the findings.

Stage one, in brief

The examination produces the `critical-guide/` package in the paper's own folder: the ledger, the annotated copies of the source, the typeset guide, the summary, and the audit trail of what was dropped and why.

That is a manual's worth of material on its own, and it has one. What matters here is what stage two consumes: `issues.yaml`, where every finding carries a tag, a grade, an anchor into the source, a note saying what is wrong, and — ideally — a request saying what would repair it.

Stage two is treated as a black box by the orchestrator, and so is stage one. Neither assumes the other's directory layout or filenames, because installed versions differ. When a stage returns, what's actually on disk is what gets worked from.

What stage two checks before starting

Four preconditions, in order. It refuses rather than improvising.

1. A **critical-guide/** directory exists, with `issues.yaml` in it. There is no repair stage without an examination stage.
2. The source named in the ledger resolves.
3. **Every anchor still matches that source.** This is the one that matters. If anchors have stopped resolving, the source has moved on since the guide was written, and repairing against a stale ledger produces changes aimed at passages that no longer exist.
4. The issues it intends to repair carry enough to work from.

Then it copies the source — the original is never touched — and works only on the copy.

Triage, before any mathematics

The first thing stage two does with the ledger is classify it, because that determines the shape of everything after.

Absence or positive error

An issue saying a proof is missing, a definition is never given, a claim is asserted and never established, is an **absence**.

An issue saying a sign is wrong, a base ring is not commutative, a statement is false as printed, is a **positive error**.

Positive errors are often repairable. Absences almost never are. Supplying the missing proof of someone's main theorem is doing the authors' work, and — the operative point — it is not recoverable from the paper. So absences route to proposals or to the report of what cannot be repaired, and no workload gets planned around them.

Related absences get consolidated rather than enumerated. Five claims missing the same calculation is one omission affecting five passages, not five findings.

Groups, not items

Two repairs interact if they touch the same object, or if one's proof cites the other's statement. The ledger's dependency field names some of these; it will not name all of them, and the ones it misses are the dangerous kind.

This matters because dispositions are assigned to *groups*. Two repairs can each be defensible alone and inconsistent together — restrict a definition to make one proof work, and every example that relied on the old generality now needs checking. Linked changes get decided together, and the disposition stays consistent across both builds.

Dependency roots are preferred, since repairing a root can retire its dependants for free, and a branch can disappear entirely once a proposed route is checked and found invalid.

Working out the repairs

For each issue it takes on: what the paper claims, what its argument establishes, what is missing between them, and the mathematics that closes the gap, **written out in full**.

“One would need to show that the adjunction extends” is not a repair. Where the gap cannot be closed, that is said precisely — and that is a result, not a failure. It’s what the report of unrepairable issues is made of.

It does not attempt everything. The set it takes on is one it can audit properly, weighted toward dependency roots and positive errors, and it says what it scoped and why. A silently truncated workload reads as a completed one.

The audit

Every repair goes to an independent skeptic asking four questions, not one:

1. Does it actually fix the stated problem?
2. Does it break anything else in the paper?
3. Is it minimal, or does it change more than it needs to?
4. Does it change what the paper claims?

One skeptic for a local repair; three for anything touching a hypothesis or a statement — scaled to what being wrong would cost. Each has to examine the proposed mathematics and its consequences in the paper; agreement based only on reading the ledger is not verification.

A repair that fails audit is recorded with its refutation rather than silently dropped.

Apply, propose, or escalate

The question is never how big the change is. Every repair changes what the paper says — fixing a misprinted subscript changes what a display asserts. The question is whether the repair is **forced**.

Disposition	When
Apply	the correction is recoverable from the paper itself: exactly one way to make the text consistent with what is demonstrably already there — the paper’s own diagram, its own proof, how a later section uses the result, the actual content of a work it cites. Nothing is being decided; what the paper already commits to is being read off.
Propose	the repair requires a decision the paper does not determine. Several defensible corrections exist, and choosing between them needs information the paper doesn’t supply. Choosing would put words in the authors’ mouths.
Escalate	no repair is available, or the choice sits upstream of most of the remaining repairs. If resolving it one way changes what a large part of the paper says, the repair is not determined, and a source that silently embodies one resolution misleads.

The test, in one sentence: *could this correction be reconstructed from the paper alone, or did something have to be decided?*

Why so few get applied

Even among forced repairs, two or three at most — unless you ask for more.

The first reason is obvious: writing into someone’s paper is a strong act and you should be the one choosing.

The second is the one that actually sets the number. **Interactions scale quadratically.** Three applied repairs give three pairs to check for coherence. Twenty give a hundred and ninety. The coherence check is only tractable at small numbers, and a pipeline that applied twenty repairs would be claiming a check it had not performed.

Everything else is proposed — rendered in position with its tag, so the marked-up build is a menu you browse and answer.

Two builds

The working copy compiles two ways, and which one you get is a command-line switch rather than a line anybody edits:

```
pdflatex -jobname paper-repaired-notes \
    paper-repaired.tex

pdflatex -jobname paper-repaired-clean \
    "\def\agenticclean{}\input{paper -
    repaired}"
```

Notes-on is the default deliberately. A file of unmarked edits that reads as the authors' own text is the dangerous artifact, so the clean build has to be asked for.

Then both are checked: the clean build contains every applied repair and no trace of any proposal; numbered statements carry the same numbers in both builds and the same as the original; no unresolved references, and no new overfull boxes beyond the original's.

Numbering is compared through the .aux files' label entries, never by scraping text out of the two PDFs. A repair that changes a sentence's length reflows the paragraph, so which statements begin a line differs between builds even when every number is identical. Comparing extracted text produces divergences that aren't there.

Corrections flow back

Finding errors in the guide is an expected output, not a surprise. Writing the mathematics out is a stronger check on a finding than the finding's own verification was, because it forces a direct comparison with the source and with every hypothesis the repair uses.

Those corrections are written to a *revised* ledger beside the original. The original is never overwritten — the difference between them is the audit trail.

Reading What Comes Back

Two packages, and a marked-up PDF that is meant to be read rather than filed.

What is on disk

```

<paper folder>/
  paper.tex           the original
  paper.pdf
  critical-guide/     stage one's
    package
      00-guide.pdf    summary +
    issue list
      issues.yaml     the ledger
      issues-revised.yaml corrections
    found
      annotated-*.pdf the paper,
    annotated
  paper-repaired.tex  the working
    copy
  agentic-edits.tex   the markup,
    copied in
  paper-repaired-notes.pdf marked up
  paper-repaired-clean.pdf applied
    changes only
  04-proposed.md      why each
    proposal

```

Start with `paper-repaired-notes.pdf`. It is the document the whole pipeline exists to produce, and everything in it is tagged, so you can move between it and the issue list without translating.

The three states

State	In the notes build	In the clean build
applied	struck-out red original, blue replacement	replacement silently in place
proposed	purple, labelled, with the replacement text shown in position	absent — it was not adopted
removed	framed and greyed, labelled	gone

Every change carries its issue tag, and every change carries a footnoted note saying why. An unexplained diff is not a repair.

Proposals show their full replacement text where the replacement would go, so you never have to open another file to learn what a proposal actually says. The reasoning behind it is in the proposals document; the text itself is in the PDF.

A proposal never consumes a statement number. Counters are frozen inside a proposal block, so a proposed replacement for Theorem 4.2 prints as Theorem 4.2 rather than pushing everything after it down by one. The numbering in both builds matches the original, which is what makes the two builds comparable at all.

Answering by tag

The hand-back names the tags, and the tags are how you reply:

Apply M-5 and M-31.

That is not a text edit, and it isn't treated as one. Each named proposal

moves to applied, and then the coherence check runs again against everything already applied, and both builds are rebuilt and re-verified.

A proposal adopted in isolation is exactly the split the grouping rule exists to prevent — the repair that was defensible on its own page and inconsistent with the one three sections later.

The clean build is not a revised paper. It is what the paper would say if these specific changes were adopted, produced so you can read the result in context rather than reading a diff. Sending it to anyone as the corrected version skips the step where a person decides, which is the step the whole pipeline is built around.

What the markup cannot do

Three limits, worth knowing before you go looking for a bug.

A proposal cannot add a bibliography entry. A bibliography item inside a proposal block is discarded in the clean build; outside it, it leaks into a build that never proposed it. Such a source is cited as plain text in the proposal and flagged.

A tag inside mathematics is boxed, so it cannot break across lines. A tag on a long display can push the line.

Classes vary. The markup has been verified against two document classes, `amsart` and `elsarticle`, covering framed theorem environments inside and outside lists, class-defined theorem environments sharing a counter, and cancellation in displays. A third class may behave differently — and the place to find that out is a trivial change, not a deep one.

What cannot be repaired at all

The third output, and the one most likely to be skipped.

It lists the issues no repair reaches: absences that would require doing the authors' work, and choices that sit upstream of everything else — where resolving the question one way or the other changes what a large part of the paper says.

That last category is the most useful thing in the package for a reader who has to form a judgment, because it says something no issue count can. An underdetermined repair is a different situation from a long list of fixable errors, and it is the situation the authors most need to hear about.

If a stage stops short

The pipeline reports exactly how far it got rather than papering over it.

If the handoff check failed — no `critical-guide/` directory, no ledger — nothing was repaired, and the report says so and says what is needed to finish.

If stage two's anchor check failed, the source has moved since the guide was written. Re-run the examination against the current version; the ledger is what is stale, not the paper.

And every quantity in the hand-back was measured. Counts come from the files. Durations come from timestamps recorded as the run passed each phase boundary, not from a figure quoted in an earlier report.

Quick Reference

Invocation

```
/write-review-and-repair <pdf path>  
/write-review-and-repair arXiv:<id>  
/write-review-and-repair <folder with the  
    paper>
```

Runs unattended, end to end. Each stage can also be run alone:

```
# stage one only  
/write-critical-guide <paper>  
  
# stage two, against an existing package  
/write-paper-repairs <paper folder>
```

The pipeline

Step	What happens
1	Examine: <code>write-critical-guide</code> writes <code>critical-guide/</code> beside the paper
2	Confirm the handoff: the directory exists and holds <code>issues.yaml</code> ; stop and report if not
3	Repair: <code>write-paper-repairs</code> consumes the ledger and writes the working copy
4	Report: where everything is, and that this is material to review rather than an accepted revision

Stage two's preconditions

#	Check
1	<code>critical-guide/</code> exists with <code>issues.yaml</code>
2	the source named in the ledger resolves
3	every anchor still matches that source — stale ledgers are refused, not worked around
4	the issues to be repaired carry a note, ideally a repair request

Dispositions

Disposition	Test
Apply	recoverable from the paper itself: exactly one way to make the text consistent with what is already there
Propose	requires a decision the paper does not determine
Escalate	no repair available, or the choice sits upstream of most of the rest

Two or three applied at most by default. Interactions scale quadratically, and the coherence check is only tractable at small numbers.

Markup states

State	Notes build	Clean build
applied	struck-out red, blue replacement	replacement in place
proposed	purple, labelled, text in position	absent
removed	framed and greyed, labelled	gone

Markup commands

```

\agenticaidition [M-5]{text}
\agenticroval [M-5]{text}

% fragile tokens: \cite, \ref, \label
\agenticrovalbox [M-5]{text}

% worked out, not adopted
\agenticroposal [M-5]{replacement}

% inside an existing display
\agenticromathadd [M-5]{t}
\agenticromathdel [M-5]{t}

% a whole new list item
\agenticroitem [M-5]{text}

% one line, footnoted
\agenticronote{why this change}

\begin{agenticroadded} [M-5]      ...
\end{agenticroadded}

\begin{agenticroremoved} [M-5]   ...

```

```
\end{agenticremoved}

\begin{agenticproposed}[M-5] ...
\end{agenticproposed}
```

Inline commands cannot carry display math, theorem environments, or anything crossing a paragraph — use the environments for those. Always carry the tag; every change gets a note saying why.

Markup rules that came out of real use

Rule	Why
Never duplicate a display to show a change inside it	the math add/delete commands work within the existing display; two whole displays for a one-glyph fix is the failure this replaced
Box the fragile token and nothing else	strike-out cannot process every citation macro, and a protective box prevents line breaks inside it; keep surrounding prose outside
Hang the tag off the last piece	so it does not force a break in the middle of a removed phrase
A proposal must not consume a number	counters are frozen; step the counter inside the block if the proposal restates a numbered result, relatively rather than absolutely
Labels are disabled inside a proposal	a label defined there resolves in the notes build and prints ?? in the clean one, making the clean build wrong

Builds

```
pdflatex -jobname paper-repaired-notes \
    paper-repaired.tex

pdflatex -jobname paper-repaired-clean \
    "\def\agenticclean{}\input{paper-repaired}"
```

Two passes each, with `bibtex` between them if the paper has a bibliography. Notes-on is the default; the clean build has to be asked for.

Then verify: the clean build carries every applied repair and no proposal; statement numbers match across both builds and the original, compared through the `.aux` label entries rather than extracted text; no unresolved references; no new overfull boxes.

Outputs

File	What it is
critical-guide/ paper-repaired.tex	stage one’s package the working copy; the original is untouched
paper-repaired-notes.pdf	changes and proposals marked up — read this one
paper-repaired-clean.pdf	applied changes silently in place
04-proposed.md	the reasoning behind each proposal
issues-revised.yaml	corrections to the ledger; never overwrites it
agenticc-edits.tex	the markup definitions, copied in

Adopting a proposal later

Name the tags — *apply* M_5 and M_{31} . Each moves to applied, the coherence check re-runs against everything already applied, and both builds are rebuilt and re-verified. Never a text edit in isolation.

Known limits

A proposal cannot add a bibliography entry — such a source is cited as plain text and flagged. A tag inside mathematics is boxed and cannot break across lines. The markup is verified against `amsart` and `elsarticle`; another class may differ, and a trivial change is where to find that out.